



Tecnológico de Monterrey

Evidence 2: Review 3

María Guadalupe Soto Acosta A00228158

Jimena Díaz Franco A01403295

Ilan Gómez Guerrero A01621122

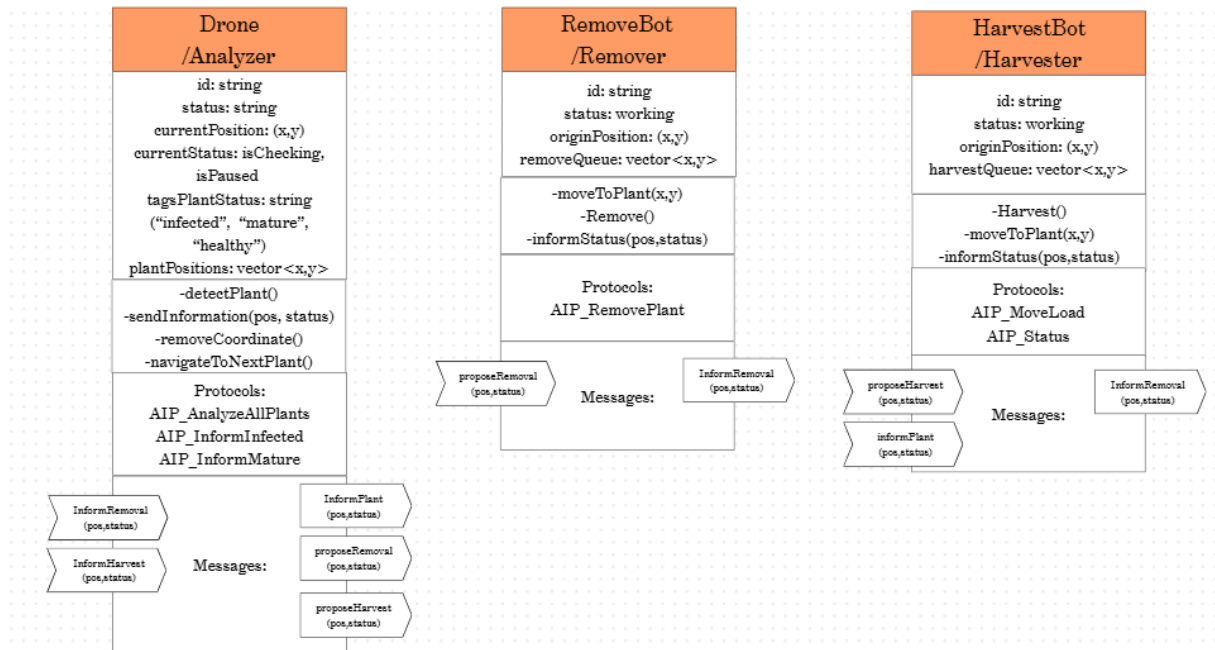
Francisco Raziel Andalón Aguayo A01640235

November 29th, 2025

Modeling of Multi-Agent Systems with Computer Graphics

Group 301

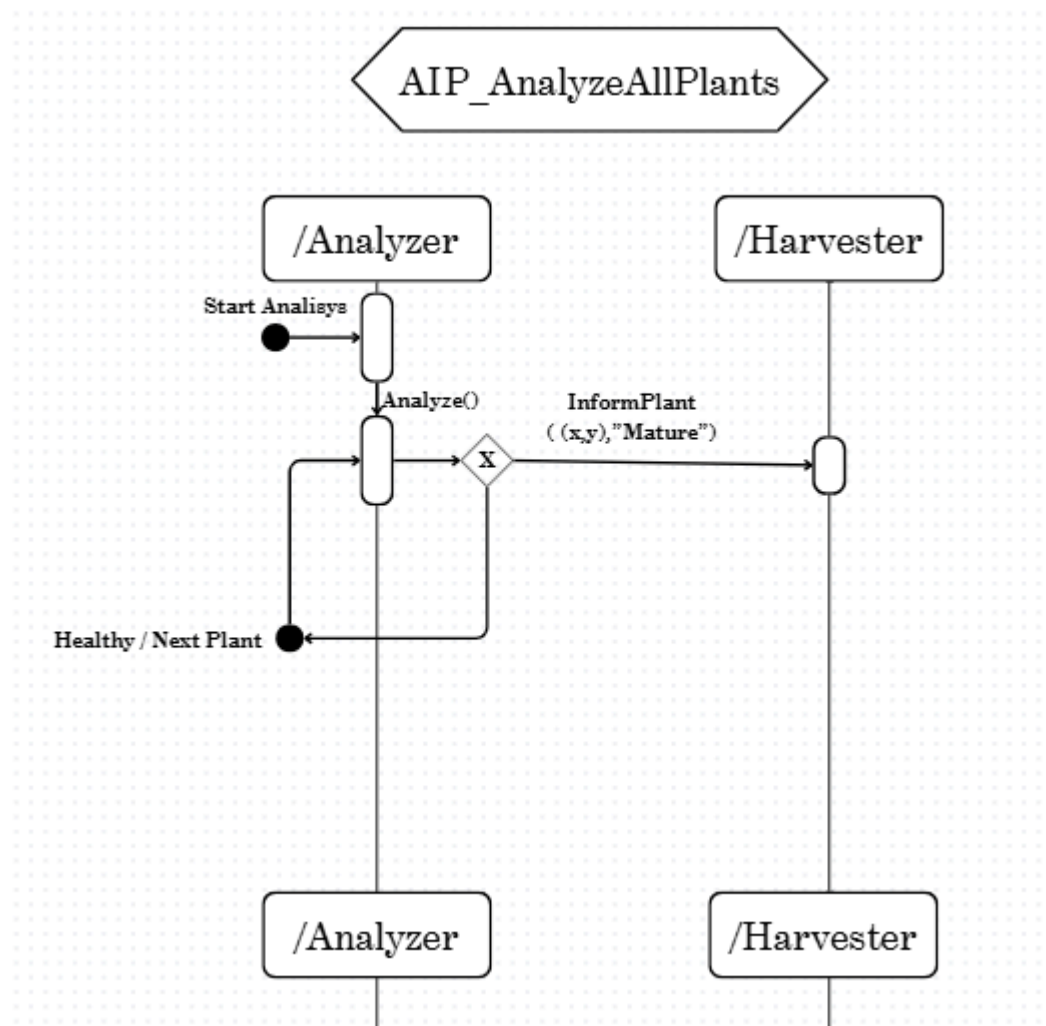
Agent Class Diagram



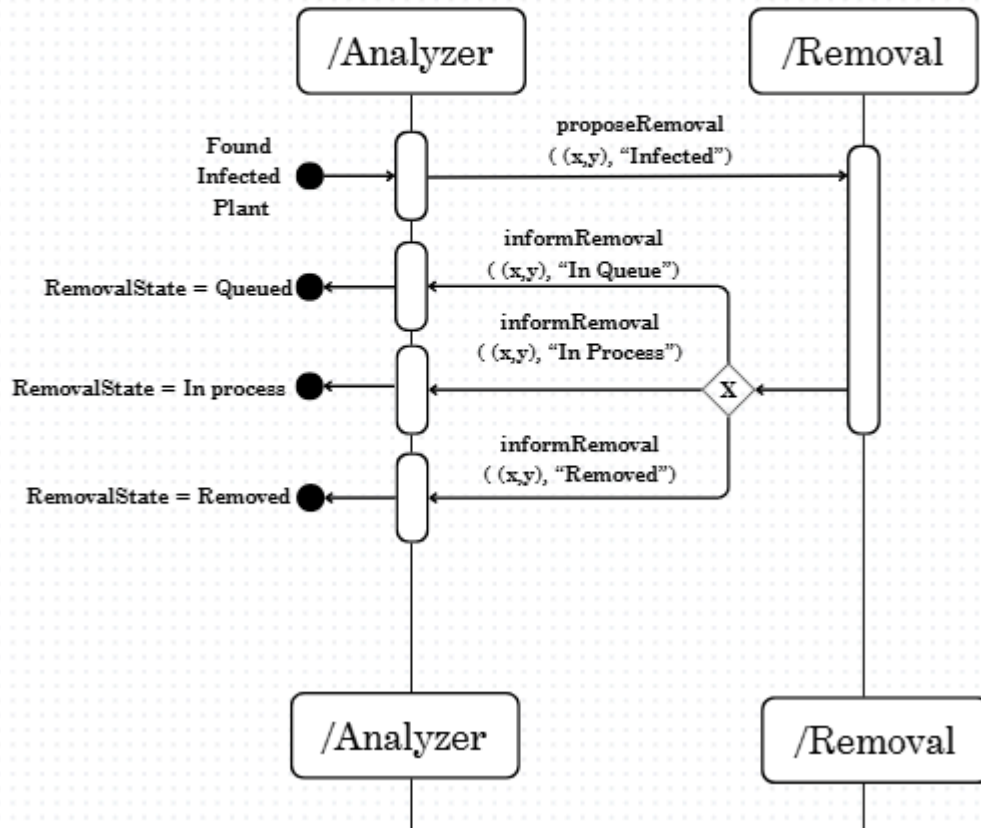
For this evidence, the agent diagram remains the same as before, and the behavior of the three agents has not changed. Each agent preserves the attributes, methods, protocols, and message types shown in the diagram. Their roles are also exactly the same: the Analyzer continues detecting infected or mature plants, the RemoveBot is still responsible for eliminating infected ones, and the HarvestBot handles the ripe plants.

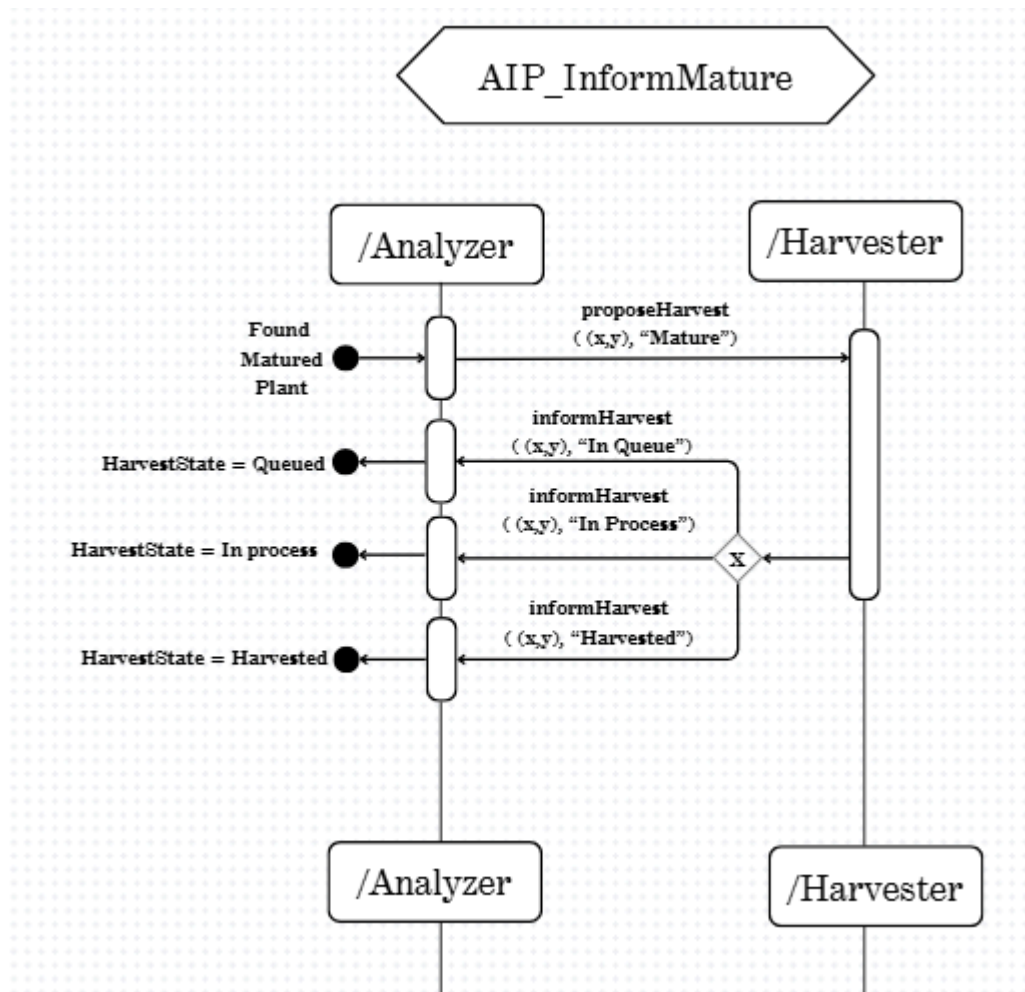
The only difference in the current Unity implementation is the way communication occurs during execution. Instead of sending formal messages such as *InformPlant*, *proposeRemoval*, or *proposeHarvest*, the interaction happens directly through Unity components and object properties. For example, when the Analyzer detects an infected plant using tags and colliders, it simply calls a method on the RemoveBot to register the plant's position. The same applies to ripe plants: once the Analyzer identifies one, it passes its position directly to the HarvestBot through a method call.

Interaction Diagram (AIP)



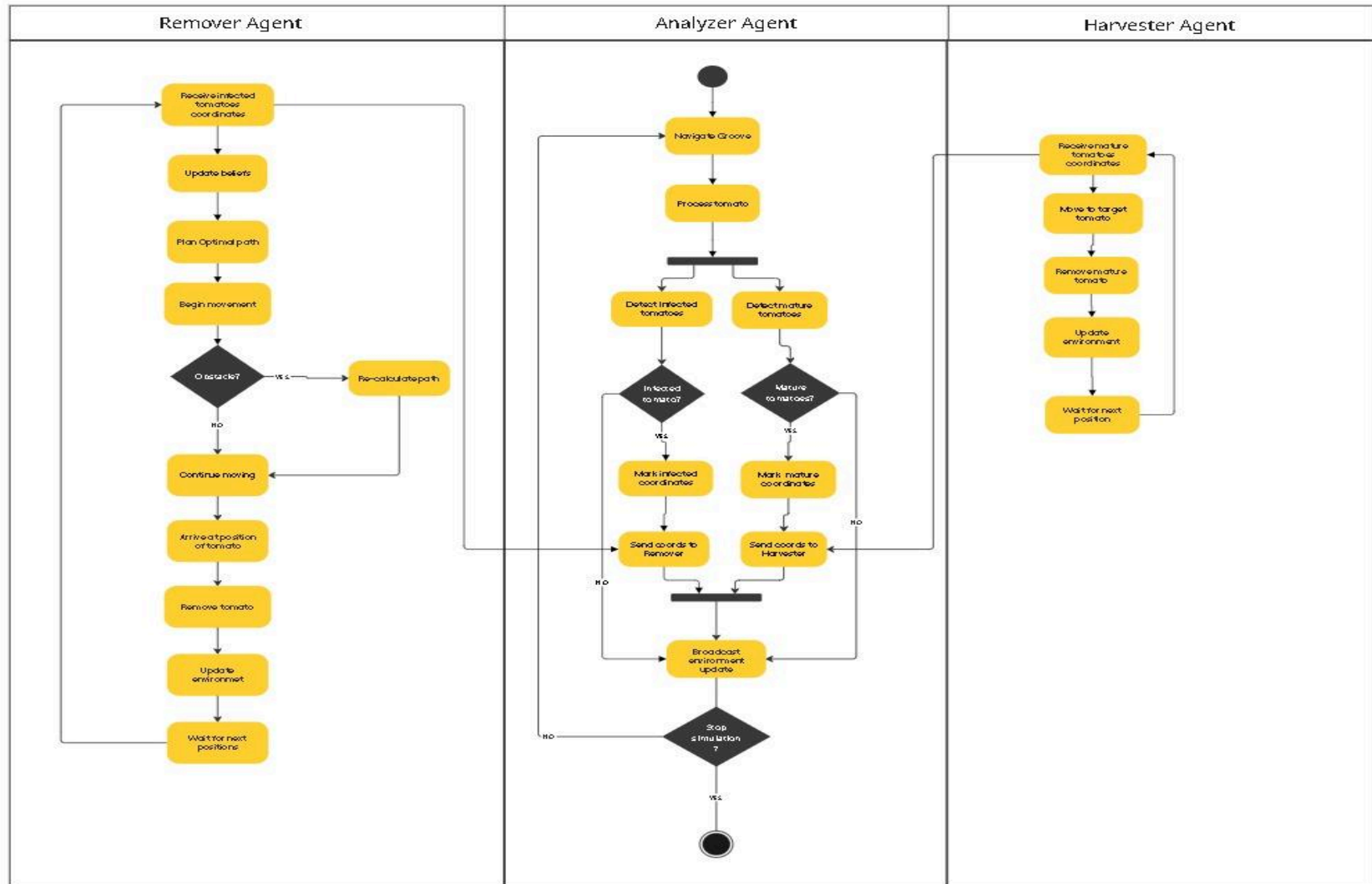
AIP_InformInfected



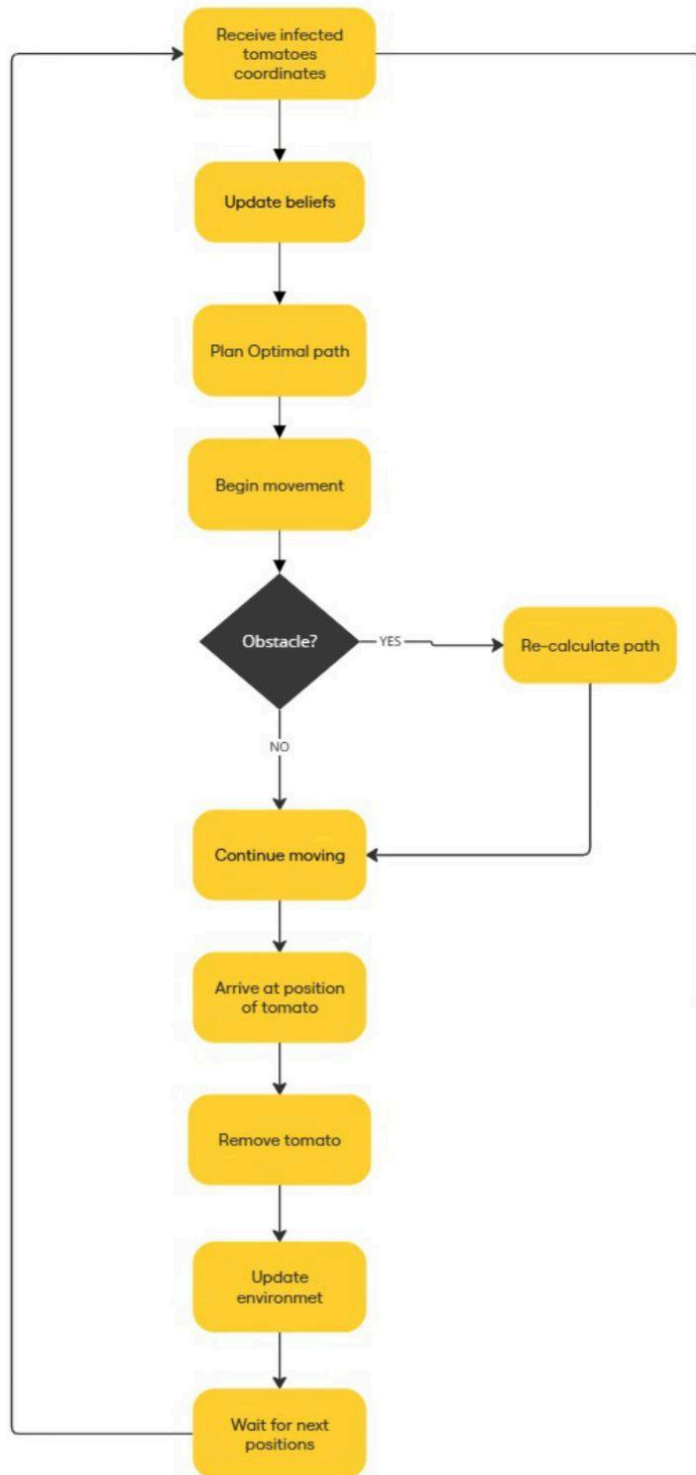


The AUML interaction and protocol diagrams remain exactly the same as in the previous version of the project. The roles of the agents and the logic of their communication do not change: the Analyzer notifies the RemoveBot when it detects an infected plant, and it notifies the HarvestBot when it detects a mature one. In return, both robots send back status updates such as “queued”, “processing”, and “removed” or “harvested”, as shown in the protocol diagrams.

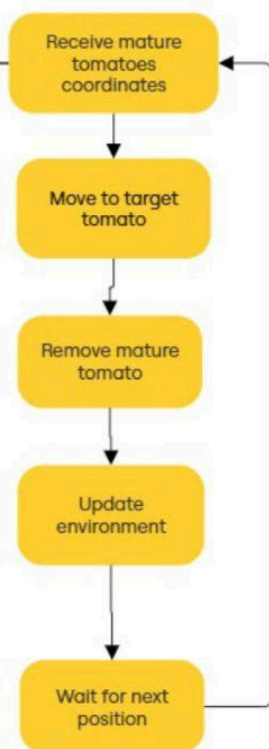
Activity Diagram



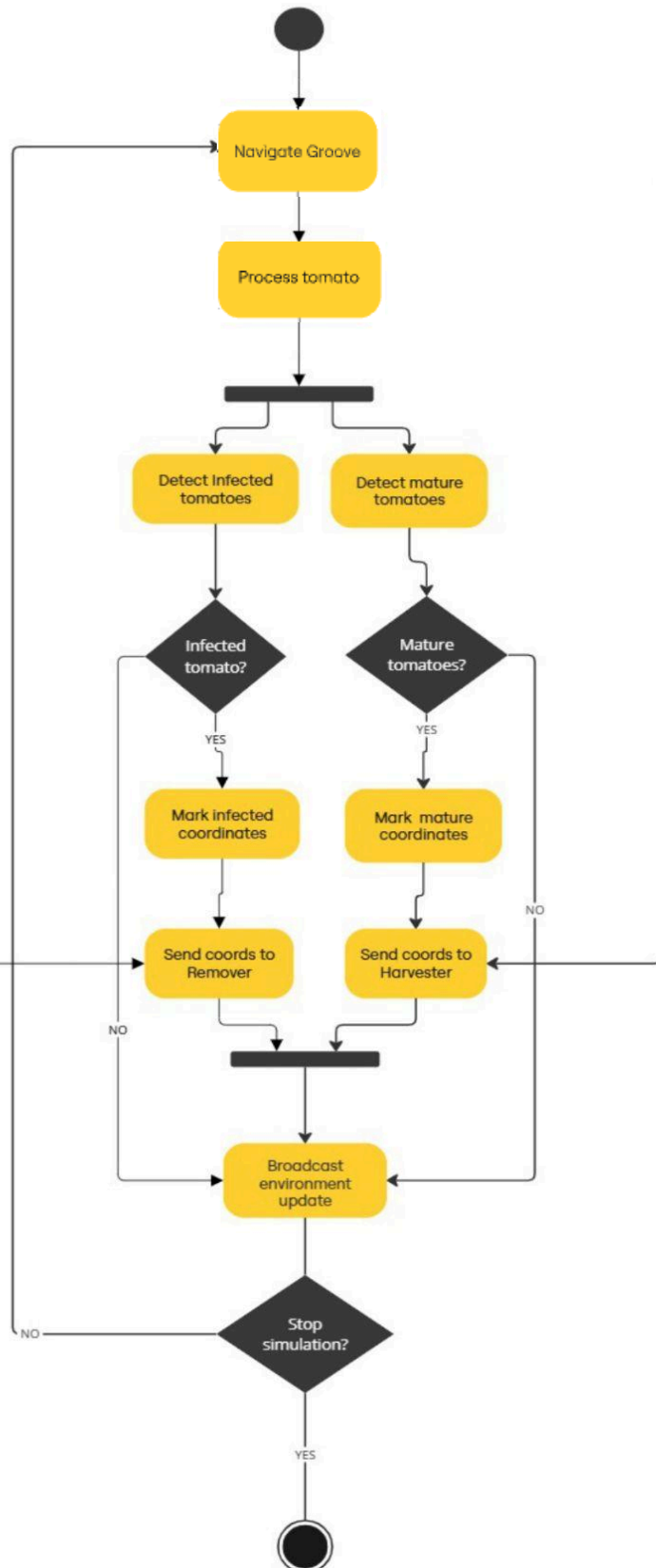
Remover Agent



Harvester Agent



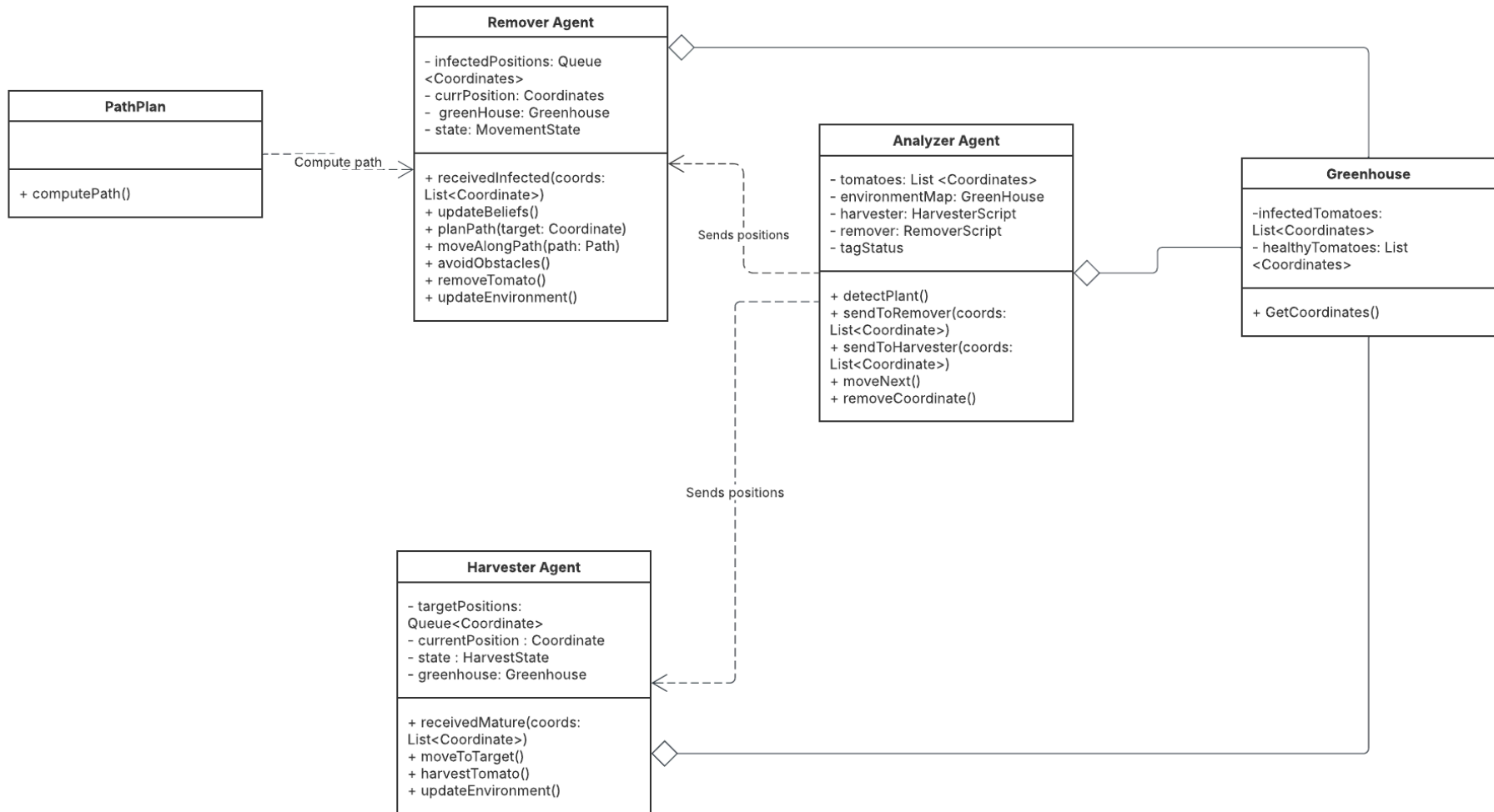
Analyzer Agent



The activity diagram remains the same as in the previous version, since the overall workflow of the system has not changed. The Analyzer still visits each plant, checks its condition, and decides whether it is infected, ripe, or healthy. The only difference in this implementation is the detection method: instead of analyzing images, the Analyzer now identifies the plant's state through Unity tags and colliders when it reaches each position.

Once a plant is detected, the flow continues exactly as before. If the Analyzer encounters an infected plant, it forwards the position to the Remover so it can remove it. If the plant is ripe, it sends the position to the Harvester so it can perform the harvest. Healthy plants simply cause the Analyzer to move to the next point without triggering any additional actions. After scanning all plants, the Analyzer checks whether the process should continue or stop, and the cycle repeats until the activity reaches its end state.

Standard Class Diagram



In the updated class diagram, the main change is the removal of the “Detection Model” component that the Analyzer originally included. In the previous version, the Analyzer had a class dedicated to processing images, detecting infected fruits, and identifying mature tomatoes using a computer-vision model. Since this functionality is not implemented in the current Unity version, that entire module was removed to better match how the system actually works.

Instead of using image models, the Analyzer now detects plants through Unity’s built-in tools, mainly colliders and object tags. Because of that, the Analyzer no longer needs attributes such as processed image data, detection methods, or a reference to a model. Its responsibility is simply to move through the greenhouse, identify plants by tag, and send their positions directly to the Remover or Harvester.

Everything else in the class diagram remains the same. The roles of the three agents did not change, and their interaction logic still follows the same flow. The only adjustment was removing the image-processing elements.

Work Plan

Task	Description	Responsible	Estimated Date	Effort estimated (hours)	Real effort taken (hours)	Status
Create GitHub repository	Set up for code and documentation	Jimena	November 10th	0.5	0.5	Done
Create communication channel	(Teams, Discord, etc.)	Paco	November 10th	0.1	0.1	Done
Define simulation environment	Build initial grid and greenhouse in Unity	Ilan	November 15th	2.0	2.0	Done
Implement Analyzer agent	Detection and marking system	Ilan	November 18th	2.0	3.0	Done
Implement Removed agent	Path planning and removal	Jimena	November 23th	2.0	3.0	Done
Implement Cosecha agent	Harvesting logic	María	November 25th	2.0	3.0	Done
Integrate and test agents	Validate agents coordination in simulation	María	December 2nd	1.0	1.0	Unfinished
Document progress	Update proposal and learning	Paco	December 4th	1.0	1.0	Unfinished

In this deliverable, as a team, one of the main things we learned was how important it is to connect theoretical agent models with the practical limitations of an actual implementation. We started with a more complex design that included image-based detection and message exchanges between agents, but once we moved to Unity, we realized that some things could be simplified using built-in tools like colliders and tags. This helped us understand that the system can keep the same logic and overall structure even if the technical details change.

We also learned how valuable it is to communicate clearly within the team. Each agent's behavior depended on the others, so we had to make sure everyone understood how the Analyzer, Remover, and Harvester should interact. Coordinating these parts taught us how to divide tasks, review each other's work, and adjust our design when something didn't work as expected.

Overall, this activity helped us improve both our technical skills and our teamwork, as we had to constantly compare our diagrams, code, and Unity implementation until all parts aligned correctly.